

JAXB Developer Guide

Java Architecture for XML Binding

Marshalling • Unmarshalling • Schema Generation • Code Generation

1. Introduction

JAXB (Java Architecture for XML Binding) is a framework that allows Java developers to map Java classes to XML representations. It provides two main functions:

- Marshalling – converting Java objects into XML
- Unmarshalling – converting XML back into Java objects

JAXB also supports schema generation from annotated classes and class generation from XSD schemas, enabling a complete round-trip workflow.

2. Core JAXB Annotations

The following annotations control how Java classes and fields map to XML:

Annotation	Purpose
@XmlRootElement	Marks a class as a top-level XML element
@XmlAccessorType	Controls which fields/methods are bound (e.g. FIELD)
@XmlAttribute	Serialises a field as an XML attribute
@XmlElement	Serialises a field as a child XML element
@XmlType	Defines the XML type name and element ordering
@XmlEnum	Maps a Java enum to an XSD enumeration type
@XmlEnumValue	Specifies the XML string value for an enum constant

3. Marshalling: Java to XML

Marshalling converts a Java object graph into XML. The example below shows a Library containing a list of Book objects being serialised.

3.1 Annotated Model Classes

```
@XmlRootElement(name = "library")
@XmlAccessorType(XmlAccessType.FIELD)
class Library {
    @XmlAttribute
    private String name;
```

```
@XmlElement(name = "book")
private List<Book> books = new ArrayList<>();
}

@XmlAccessorType(XmlAccessType.FIELD)
class Book {
    @XmlAttribute          private String isbn;
    @XmlElement            private String title;
    @XmlElement            private String author;
    @XmlElement            private int    year;
}
```

3.2 Marshalling Code

```
JAXBContext ctx = JAXBContext.newInstance(Library.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

StringWriter writer = new StringWriter();
m.marshal(library, writer);
System.out.println(writer.toString());
```

3.3 Sample XML Output

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<library name="City Central Library">
  <book isbn="978-0-13-468599-1">
    <title>Effective Java</title>
    <author>Joshua Bloch</author>
    <year>2018</year>
  </book>
</library>
```

4. Unmarshalling: XML to Java

Unmarshalling parses an XML document and recreates the Java object graph. It is the reverse of marshalling.

```
JAXBContext ctx = JAXBContext.newInstance(Library.class);
Unmarshaller u = ctx.createUnmarshaller();
Library restored = (Library) u.unmarshal(new StringReader(xml));
```

```
System.out.println("Library: " + restored.getName());
restored.getBooks().forEach(System.out::println);
```

Both operations together form a complete round-trip: Java object → XML string → Java object.

5. Schema Generation: Classes to XSD

JAXB can generate an XSD schema directly from your annotated classes using `JAXBContext.generateSchema()`. This is useful for publishing a contract or validating XML from other systems.

5.1 Additional Annotations for Richer Schemas

Annotation / Property	Schema Effect
<code>@XmlAttribute(required=true)</code>	<code>use="required"</code> on the XSD attribute
<code>@XmlElement(required=true)</code>	<code>minOccurs="1"</code> (mandatory element)
<code>@XmlElement(defaultValue="0")</code>	<code>default="0"</code> on the element
<code>@XmlType(propOrder={...})</code>	Fixes element order in <code><xs:sequence></code>
<code>@XmlEnum + @XmlEnumValue</code>	Generates <code><xs:simpleType></code> with enumeration restrictions

5.2 Schema Generation Code

```
JAXBContext ctx = JAXBContext.newInstance(Library.class, Book.class);

ctx.generateSchema(new SchemaOutputResolver() {
    @Override
    public Result createOutput(String ns, String fileName) {
        StringWriter sw = new StringWriter();
        StreamResult result = new StreamResult(sw);
        result.setSystemId(fileName);
        return result;
    }
});
```

5.3 Sample Generated XSD

```
<xs:complexType name="book">
```

```
<xs:sequence>
  <xs:element name="title" type="xs:string"/>
  <xs:element name="author" type="xs:string"/>
  <xs:element default="0" name="year" type="xs:int"/>
  <xs:element minOccurs="0" name="genre" type="genre"/>
</xs:sequence>
<xs:attribute name="isbn" type="xs:string" use="required"/>
</xs:complexType>
```

5.4 Saving to File

To write the schema to disk instead of a `StringWriter`, use a `StreamResult` backed by a `File`:

```
ctx.generateSchema(new SchemaOutputResolver() {
    @Override
    public Result createOutput(String ns, String fileName) {
        StreamResult result = new StreamResult(new File(fileName));
        result.setSystemId(fileName);
        return result;
    }
});
```

6. Class Generation: XSD to Java

The `xjc` tool (XML Schema Compiler) ships with JAXB and generates fully annotated Java classes from an XSD schema. This is the reverse of schema generation.

6.1 Running xjc

```
# Basic usage
xjc library.xsd

# Specify output directory and package
xjc -d src/main/java -p com.example.library library.xsd
```

Flag	Purpose
-d <dir>	Root directory for generated source files
-p <pkg>	Java package name for generated classes
-b <bindings>	Custom bindings file (.xjb) for renaming / type overrides
-nv	Skip strict schema validation

-verbose	Show detailed output during generation
----------	--

6.2 Generated Files

For the library schema, xjc produces:

- Library.java – @XmlRootElement class with a List<Book>
- Book.java – @XmlAccessorType class with all mapped fields
- Genre.java – @XmlEnum enumeration with @XmlEnumValue constants
- ObjectFactory.java – @XmlRegistry factory for creating instances
- package-info.java – binds the namespace URI to the package

6.3 Custom Bindings

A .xjb bindings file lets you control class names, field types, and other generation details:

```
<jxb:bindings xmlns:xs="http://www.w3.org/2001/XMLSchema"
              xmlns:jxb="https://jakarta.ee/xml/ns/jaxb"
              version="3.0" schemaLocation="library.xsd">

  <!-- Rename the generated Library class -->
  <jxb:bindings node="//xs:complexType[@name='Library']">
    <jxb:class name="BookLibrary"/>
  </jxb:bindings>

  <!-- Map year to Integer instead of int -->
  <jxb:bindings node="//xs:complexType[@name='Book']
                /xs:sequence/xs:element[@name='year']">
    <jxb:property><jxb:baseType name="java.lang.Integer"/></jxb:property>
  </jxb:bindings>
</jxb:bindings>

xjc -d src/main/java -p com.example.library -b bindings.xjb library.xsd
```

6.4 Maven Plugin

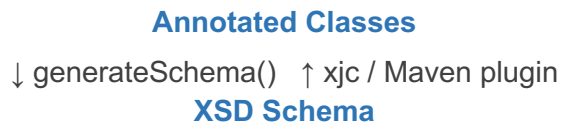
Using the Maven plugin generates classes automatically at build time, avoiding the manual xjc step:

```
<plugin>
  <groupId>org.glassfish.jaxb</groupId>
```

```
<artifactId>jaxb-maven-plugin</artifactId>
<version>4.0.5</version>
<executions>
  <execution><goals><goal>generate</goal></goals></execution>
</executions>
<configuration>
  <schemaDirectory>src/main/resources/xsd</schemaDirectory>
  <generateDirectory>target/generated-sources/jaxb</generateDirectory>
  <generatePackage>com.example.library</generatePackage>
</configuration>
</plugin>
```

7. The Complete Round-Trip

JAXB supports a full lossless round-trip between Java classes and XSD schemas:



You can go from annotated classes to XSD and back to classes repeatedly, with consistent results each time.

8. Maven Dependencies

Add these to your pom.xml to use JAXB with Jakarta EE (Java 11+):

```
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>4.0.5</version>
</dependency>
```

For Java 8, JAXB is bundled with the JDK — no dependency is needed. Change imports from `jakarta.xml.bind.*` to `javax.xml.bind.*`.

9. JAXB in JAX-RS Web Applications

This section covers using JAXB with JAX-RS REST services on the Java 11 / Tomcat 9 / JAX-RS 2.1 stack. Because JAXB was removed from the JDK in Java 11, all JAXB classes must be added as explicit Maven dependencies. The JAX-RS 2.1 reference implementation is Jersey 2.29+, which uses the `javax.ws.rs` namespace.

9.1 Java 11 and the Missing JAXB

In Java 8 and earlier, JAXB was part of the JDK. From Java 9 it was deprecated, and in Java 11 it was removed entirely via JEP-320. This means any project running on Java 11 must declare JAXB as an explicit dependency — including JAX-RS web applications deployed to Tomcat 9.

- `javax.xml.bind.*` classes are not on the classpath by default in Java 11
- The `--add-modules java.xml.bind` workaround from Java 9/10 does not work in Java 11
- Tomcat 9 does not bundle JAXB — it must be packaged inside your WAR

9.2 Maven Dependencies

Use the `javax` namespace versions (not `jakarta`) for Tomcat 9 / JAX-RS 2.1 compatibility. Jersey 2.29 is the minimum version with full Java 11 support.

```
<!-- JAX-RS 2.1 API -->
<dependency>
  <groupId>javax.ws.rs</groupId>
  <artifactId>javax.ws.rs-api</artifactId>
  <version>2.1.1</version>
  <scope>provided</scope>
</dependency>

<!-- Jersey 2.x servlet container integration -->
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet</artifactId>
  <version>2.35</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.inject</groupId>
  <artifactId>jersey-hk2</artifactId>
  <version>2.35</version>
</dependency>

<!-- JAXB API (removed from JDK in Java 11) -->
<dependency>
  <groupId>javax.xml.bind</groupId>
```



```
<artifactId>jaxb-api</artifactId>
<version>2.3.1</version>
</dependency>

<!-- JAXB Runtime implementation -->
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
  <version>2.3.6</version>
</dependency>

<!-- Jersey JAXB media provider (XML <-> Java auto-mapping) -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-jaxb</artifactId>
  <version>2.35</version>
</dependency>
```

9.3 Application Configuration

Tomcat 9 is a servlet container, not a full Java EE server, so JAX-RS support must be bootstrapped manually. There are two approaches: a JAX-RS Application class (preferred), or a `web.xml` servlet mapping.

Option A — Application Class (no `web.xml` required)

```
package com.example;

import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api") // all REST endpoints live under /api/*
public class RestApplication extends Application {
    // Jersey scans the classpath automatically;
    // return empty sets to trigger that behaviour
}
```

Option B — `web.xml` Servlet Mapping

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
  <servlet>
    <servlet-name>jersey</servlet-name>
    <servlet-class>
      org.glassfish.jersey.servlet.ServletContainer
    </servlet-class>
```

```
<init-param>
  <param-name>jersey.config.server.provider.packages</param-name>
  <param-value>com.example.resources</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>jersey</servlet-name>
  <url-pattern>/api/*</url-pattern>
</servlet-mapping>
</web-app>
```

9.4 Annotating Domain Classes for JAX-RS

JAX-RS uses JAXB annotations to automatically serialise and deserialise request/response bodies. Any class returned from or accepted by a resource method should carry `@XmlRootElement`.

```
package com.example.model;

import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlAttribute;

@XmlRootElement(name = "book")    // enables XML and JSON mapping
public class Book {

    @XmlAttribute
    private String isbn;

    @XmlElement
    private String title;

    @XmlElement
    private String author;

    @XmlElement
    private int year;

    // no-arg constructor required by JAXB
    public Book() {}

    public Book(String isbn, String title, String author, int year) {
        this.isbn = isbn; this.title = title;
        this.author = author; this.year = year;
    }
}
```

```
        // getters and setters ...  
    }
```

9.5 Writing a JAX-RS Resource Class

Resource classes map HTTP methods and paths to Java methods. Jersey automatically marshals the return value to XML or JSON depending on the Accept header, using JAXB under the hood.

```
package com.example.resources;  
  
import com.example.model.Book;  
import javax.ws.rs.*;  
import javax.ws.rs.core.MediaType;  
import javax.ws.rs.core.Response;  
import java.util.*;  
  
@Path("/books")  
public class BookResource {  
  
    // GET /api/books - returns XML or JSON list  
    @GET  
    @Produces({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })  
    public List<Book> getAllBooks() {  
        return List.of(  
            new Book("978-0-13-468599-1", "Effective Java",  
                "Joshua Bloch", 2018)  
        );  
    }  
  
    // GET /api/books/{isbn}  
    @GET  
    @Path("/{isbn}")  
    @Produces({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })  
    public Book getBook(@PathParam("isbn") String isbn) {  
        return new Book(isbn, "Effective Java", "Joshua Bloch", 2018);  
    }  
  
    // POST /api/books - accepts XML or JSON body  
    @POST  
    @Consumes({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })  
    @Produces({ MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON })  
    public Response createBook(Book book) {  
        // persist book ...  
        return Response.status(Response.Status.CREATED)  
            .entity(book).build();  
    }  
}
```

```
}
```

9.6 Serving Both XML and JSON

Jersey's JAXB media provider handles XML automatically. For JSON, add the Jersey MOXy or Jackson provider. MOXy is the simplest choice as it re-uses JAXB annotations directly.

```
<!-- MOXy JSON provider (reuses JAXB annotations for JSON) -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
  <version>2.35</version>
</dependency>
```

MOXy is auto-discovered by Jersey. No additional registration code is needed. The same `@XmlRootElement` annotated class will be serialised to XML when the client sends `Accept: application/xml` and to JSON when it sends `Accept: application/json`.

Alternatively: Jackson JSON Provider

```
<!-- Jackson JSON provider -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-jackson</artifactId>
  <version>2.35</version>
</dependency>
```

With Jackson, register the feature explicitly in your Application class:

```
import org.glassfish.jersey.jackson.JacksonFeature;

@ApplicationPath("/api")
public class RestApplication extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(JacksonFeature.class);
    }
}
```

9.7 Exception Mapping with JAXB

JAX-RS ExceptionMappers convert Java exceptions into HTTP responses. Combining them with JAXB lets you return structured XML or JSON error bodies automatically.

```
// JAXB-annotated error response body
@XmlRootElement(name = "error")
public class ErrorResponse {
    @XmlElement private int    code;
    @XmlElement private String message;
    public ErrorResponse() {}
    public ErrorResponse(int code, String message) {
        this.code = code; this.message = message;
    }
}

// ExceptionMapper that produces a JAXB body
@Provider
public class NotFoundMapper
    implements ExceptionMapper<NotFoundException> {

    @Override
    public Response toResponse(NotFoundException ex) {
        return Response
            .status(Response.Status.NOT_FOUND)
            .entity(new ErrorResponse(404, ex.getMessage()))
            .type(MediaType.APPLICATION_XML)
            .build();
    }
}
```

9.8 Project Structure and Deployment

The finished project follows standard Maven WAR layout. The WAR is dropped into Tomcat's webapps directory for deployment.

Path	Purpose
src/main/java/com/example/RestApplication.java	@ApplicationPath entry point
src/main/java/com/example/resources/BookResource.java	JAX-RS resource class
src/main/java/com/example/model/Book.java	@XmlRootElement domain class
src/main/resources/	Application properties / logging config
src/main/webapp/WEB-INF/web.xml	Optional servlet mapping (Option B only)
pom.xml	Maven build with Jersey + JAXB dependencies

Build and deploy:

```
mvn clean package                    # produces target/myapp.war
cp target/myapp.war $CATALINA_HOME/webapps # deploy to Tomcat 9
```

Once deployed, the API is available at:

```
GET http://localhost:8080/myapp/api/books      (returns XML or JSON)
GET http://localhost:8080/myapp/api/books/{isbn}
POST http://localhost:8080/myapp/api/books     (body: XML or JSON Book)
```

9.9 Version Compatibility Summary

Component	Version	Notes
Java	11	JAXB removed – must be added as dependency
Tomcat	9.x	Servlet 4.0 container; does not bundle JAXB or JAX-RS
JAX-RS	2.1 (JSR 370)	Uses javax.ws.rs.* namespace
Jersey	2.35	JAX-RS 2.1 RI; 2.29+ required for Java 11 support
JAXB API	javax.xml.bind 2.3.1	javax namespace (not jakarta) for this stack
JAXB Runtime	org.glassfish.jaxb 2.3.6	Implementation; packaged inside WAR
JSON provider	jersey-media-moxy 2.35	Or jersey-media-json-jackson 2.35

10. Returning a List as XML — Employee Example

Returning a `List<Employee>` directly from a JAX-RS method does not work cleanly with JAXB. Because JAXB requires a single `@XmlRootElement` as the top-level element, a bare generic list has no root element name and Jersey will throw a `MessageBodyWriter not found` error at runtime. The solution is a dedicated wrapper class.

10.1 The javax Namespace

All JAXB and JAX-RS imports on the Java 11 / Tomcat 9 / JAX-RS 2.1 stack must use the `javax` namespace, not `jakarta`. The `jakarta` namespace belongs to Jakarta EE 9+, Tomcat 10+, and Jersey 3.x.

Namespace	Stack
<code>javax.xml.bind</code> + <code>javax.ws.rs</code>	Java 11, Tomcat 9, JAX-RS 2.1, Jersey 2.x ← this stack
<code>jakarta.xml.bind</code> + <code>jakarta.ws.rs</code>	Jakarta EE 9+, Tomcat 10+, JAX-RS 3.x, Jersey 3.x

Correct imports for each class:

```
// Employee.java and EmployeeList.java
import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlAttribute;

// EmployeeResource.java
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.GenericEntity; // if using GenericEntity approach
```

10.2 The Employee Model

```
package com.example.model;

import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlAttribute;

@XmlRootElement(name = "employee")
```

```
public class Employee {

    @XmlAttribute
    private int id;

    @XmlElement
    private String firstName;

    @XmlElement
    private String lastName;

    @XmlElement
    private String department;

    @XmlElement
    private double salary;

    public Employee() {} // required by JAXB

    public Employee(int id, String firstName, String lastName,
                    String department, double salary) {
        this.id = id;  this.firstName = firstName;
        this.lastName = lastName;  this.department = department;
        this.salary = salary;
    }
    // getters and setters...
}
```

10.3 The Wrapper Class

Because JAXB cannot marshal a raw `List<Employee>`, a wrapper class with `@XmlRootElement` is used to provide the enclosing XML root element.

```
package com.example.model;

import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.XmlElement;
import java.util.List;
import java.util.ArrayList;

@XmlRootElement(name = "employees") // becomes the XML root element
public class EmployeeList {

    @XmlElement(name = "employee") // each item becomes <employee>
    private List<Employee> employees = new ArrayList<>();
}
```



```
public EmployeeList() {}           // required by JAXB

public EmployeeList(List<Employee> employees) {
    this.employees = employees;
}

public List<Employee> getEmployees() { return employees; }
}
```

10.4 The JAX-RS Resource Method

```
package com.example.resources;

import com.example.model.Employee;
import com.example.model.EmployeeList;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import java.util.List;

@Path("/employees")
public class EmployeeResource {

    @GET
    @Produces(MediaType.APPLICATION_XML)
    public Response getAllEmployees() {

        List<Employee> list = List.of(
            new Employee(1, "Alice", "Smith", "Engineering", 72000),
            new Employee(2, "Bob", "Jones", "Marketing", 58000),
            new Employee(3, "Carol", "Williams", "Engineering", 81000)
        );

        return Response.ok(new EmployeeList(list)).build();
    }
}
```

10.5 XML Response

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<employees>
  <employee id="1">
```

```
<firstName>Alice</firstName>
<lastName>Smith</lastName>
<department>Engineering</department>
<salary>72000.0</salary>
</employee>
<employee id="2">
  <firstName>Bob</firstName>
  <lastName>Jones</lastName>
  <department>Marketing</department>
  <salary>58000.0</salary>
</employee>
</employees>
```

10.6 Approach Comparison

Approach	Root Element	Reliability
Return List<Employee> directly	None – breaks at runtime	Fails
EmployeeList wrapper class	<employees> – full control	Recommended
GenericEntity<List<Employee>>	Partial – provider dependent	Use with caution